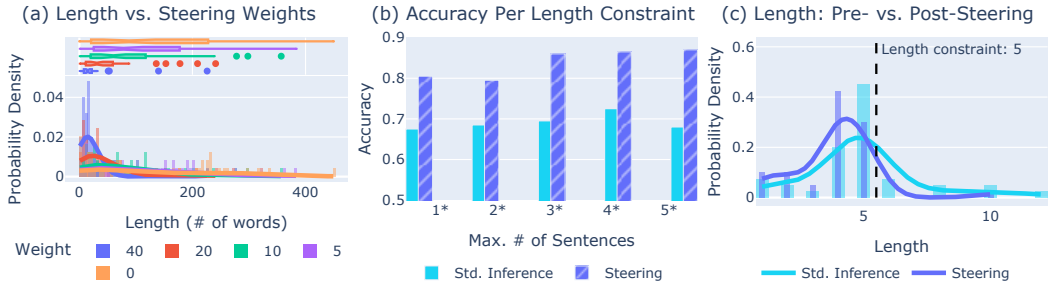

Figure 5: **Length Instructions.** (a) Modulating the steering weight c effectively adjusts response length, with larger values leading to more concise outputs. (b) Steering enhances adherence to the maximum length constraints when they are explicitly specified in the input text. (c) Shift in the response length distribution upon steering for outputs constrained to a maximum of 5 sentences.

Table 2: **Example of Steering for Length Control.** Responses to the same prompt with increasing steering weights, resulting in progressively shorter outputs in terms of sentence and word count.

Weight	Resp. to "Write a movie plot that involves dream, fist fighting, and superpower."	# of Sents.	# of Words
0	In the bustling city of New Haven, a young, introverted artist named Alex dreams of a world where his art comes to life. One night, Alex has a vivid dream where he discovers he possesses a unique superpower: the ability to bring [...]	15	324
5	In a world where dreams can be harnessed as a source of power, a young, introverted artist named Leo discovers he has the unique ability to enter and manipulate the dreams of others [...]	9	228
10	In a world where dreams can manifest into reality, a young woman named Elara discovers she possesses the rare ability to fight with her fists in her dreams [...]	10	203
20	In a world where dreams can manifest reality, a young girl named Lila discovers she can unleash her latent superpower through her dreams [...]	3	81
40	A young girl dreams of a superpower, fights a rival in a dream, and unleashes her power to save her village from destruction.	1	23

ing more general notions of *conciseness* and *verbosity*. To achieve this, we compute vector representations for instructions that prompt the model to be brief (e.g., "The answer should be brief") or to provide more detailed responses (e.g., "Provide a long answer"). We synthetically generate these prompts by appending such instructions to base queries from IFEval. After computing these steering vectors, we evaluate the model on a separate set of IFEval base queries, this time appending length instructions specified in terms of the number of sentences (e.g., "Answer using at most 3 sentences").

Steering Results. Unlike format instructions, steering the model for length constraints allows for continuous modulation, enabling interpolation between varying degrees of conciseness or verbosity. To explore this, we manually adjust the steering weight c and examine how this affects the response length. On a set of 50 base prompts without any explicit length instructions, we generate responses from Phi-3 using different values of c, measuring the distribution of output lengths. As shown in Figure 5a, increasing the value of c effectively shortens the model's responses: larger steering weights produce increasingly concise outputs. We provide an example of different outputs generated on the same inputs with different steering weights in Table 2. Next, we assess whether steering for length is effective even when the model is provided explicit length instructions. Using a set of 200 base prompts, we introduce instructions that request the model to limit its response to a maximum of $n \in \{1, \ldots, 5\}$ sentences. First, we evaluate how often the model adheres to this constraint without steering (light blue bars in Figure 5b). Then, on the same inputs, we apply the steering vector for conciseness with a weight of $c = 20$ (dark blue bars in Figure 5b). Across all five values of n, we observe a significant and consistent improvement in how often the model's responses comply with the length constraint. Figure 5c further illustrates how steering shifts the response length distribution toward shorter outputs while still allowing for variability and avoiding overly short responses. We also conducted experiments with steering for longer outputs and for exact output length, observing positive results (Appendix J). Finally, response quality evaluation results are provided in Appendix F.

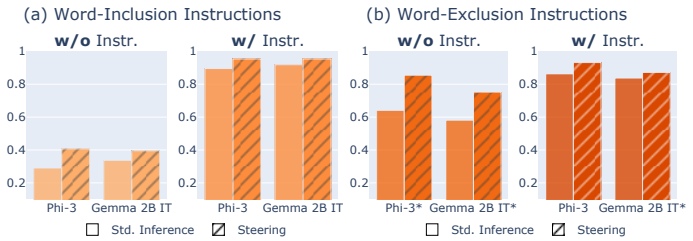

Figure 6: **Word-specific Instructions.** (a) Steering improves keyword inclusion accuracy for Phi-3 and Gemma 2 2B IT, with and without explicit instructions. (b) Negative steering reduces the occurrence of undesired keywords, in both settings.

Table 3: **Example of Word Exclusion.** The instruction alone fails to exclude "step," but steering successfully removes it.

Steer	Resp. to "What are the steps to be followed for the documentation of a GM in SAP? Do not use the word *step*."
No	Documenting a General Master in SAP involves [provides guide] By following these **steps**, you can create comprehensive [...]
Yes	## Documenting a General Master (GM) in SAP: [provides guide] By following this **comprehensive guide**, you can create a well-structured [...]

5 Word-specific Instructions

Word Inclusion. For this set of instructions, we compute word-specific steering vectors. To generate these vectors, we append different phrasings of a request to include a specific word w (e.g., "please include the word $\{w\}$ in your response") to a set of base queries. These prompts are used to compute a vector representation for the "include word $\{w\}$" instruction. While this requires a separate steering vector for each keyword at inference time, the vectors can be generated on-the-fly using arbitrary base queries unrelated to the keyword itself. Furthermore, these vectors can be computed with a small number of examples; in our experiments, we use only 20 examples. The models are then evaluated on the subset of IFEval that contains keyword inclusion/exclusion constraints.[5] The results, presented in Figure 6a, demonstrate the effectiveness of steering for word inclusion. By applying the word-specific steering vectors, we observe a notable increase in the frequency with which the model successfully includes the requested keywords in its responses.

Word Exclusion. For keyword exclusion, we initially used the same procedure, appending instructions like "ensure the word w does not appear in the answer" to base queries and computing the corresponding steering vectors. However, upon inspecting the steering vectors, we found that projecting these vectors onto the vocabulary space resulted in high logit values for the tokens corresponding to the words meant to be excluded. In other words, adding these vectors to the model's residual stream actually increased the probability of generating the very keywords we intended to exclude. To address this issue, instead of computing exclusion vectors, we compute the vectors for inclusion and then subtract them from the model's residual stream. This technique effectively steers the model away from using the words captured by the vector. Figure 6b shows the results of steering for keyword exclusion using this approach. We observe that subtracting the inclusion vectors significantly reduces the frequency of the undesired keywords in the model's responses, both in cases where no textual instructions are present and where explicit exclusion instructions are provided in the input. Table 3 reports an example where the instruction to exclude a specific word is ineffective on its own, but applying steering successfully enforces the constraint.

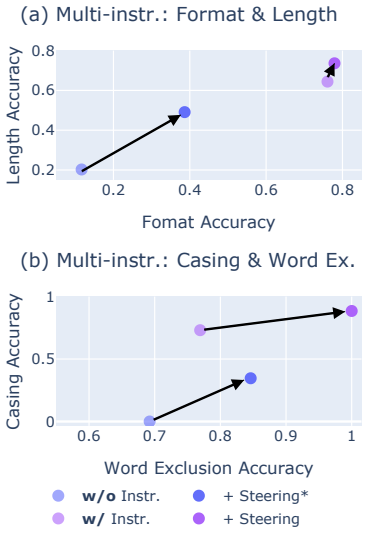

Figure 7: **Multi-instruction Steering.** Steering for two instructions improves adherence to both.

6 Multi-instruction Steering

Next, we present results on using our steering approach to handle multiple instructions simultaneously. With the same method as in the previous experiments, we steer the Phi-3 model for two

[5] IFEval prompts may contain instructions for the inclusion or exclusion of multiple keywords in the same example. We separate such cases into inputs with single-keyword instructions.

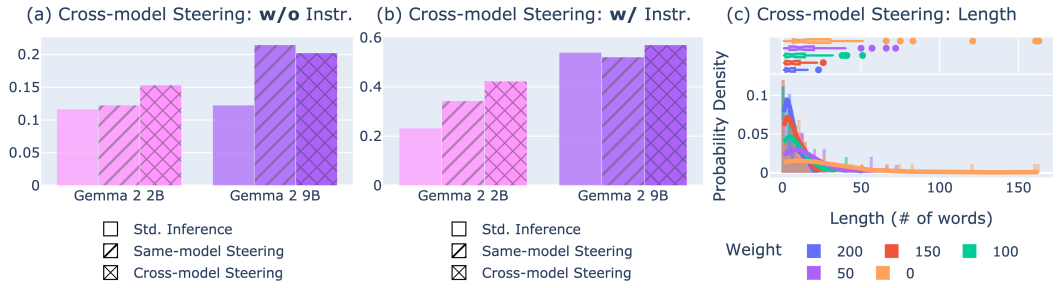

Figure 8: **Cross-model Steering.** (a,b) Steering vectors from instruction-tuned models transfer effectively to base models, with cross-model steering outperforming same-model steering in Gemma 2 2B. (c) Cross-model steering for length instructions shortens output as steering weight increases.

instructions at once, applying the steering vectors at the layers previously identified as optimal for each individual instruction. We opt for injecting multiple steering vectors simultaneously at different locations instead of combining multiple steering vectors into a single one, as previous work has shown the latter approach to be largely unsuccessful (van der Weij et al., 2024).

Figure 7 shows results from experiments with steering two instructions the same time: format (all 13 instructions) and length (Figure 7a), as well as lowercase and keyword exclusion (Figure 7b). In both cases, steering leads to improvements on both axes. These findings suggest that steering for multiple instructions at once is feasible and can lead to performance gains across different constraints. However, we anticipate that issues may arise in certain cases, particularly when dealing with conflicting instructions, where further refinement may be necessary to balance competing constraints.

7 Cross-model Steering

Instruction-tuned models show improved instruction-following abilities, suggesting they may also form more effective instruction representations. This raises the question: can we leverage the representations computed on instruction-tuned models to steer a base model more effectively? This idea is motivated by prior work showing that fine-tuning reinforces existing mechanisms in language models (Prakash et al., 2024) and that instruction tuning causes minimal weight changes (Lee et al., 2024; Jain et al., 2024). Additionally, linear representations derived through mean activation difference and sparse autoencoders (Huben et al., 2024) have been shown to be transferable between base and chat models (Panickssery et al., 2024; Kissane et al., 2024). To study this in our setting, we conduct experiments with the base and instruction-tuned versions of Gemma 2 2B and 9B. We focus on format instructions, using the same data and procedures outlined in §3. The key difference is that we apply steering vectors computed on the instruction-tuned models to steer the base models, comparing this approach to same-model steering (vectors computed on the base model).

In both the base query-only setting (Figure 8a) and with explicit instructions (Figure 8b), steering base models using vectors computed on instruction-tuned counterparts yields performance gains that demonstrate the transferability of instruction representations. Notably, in the case of Gemma 2 2B, cross-model steering outperforms same-model steering across both settings, highlighting its potential for more effective instruction adherence in smaller models. We also explore cross-model steering for length instructions. As described in §4, we compute a steering vector for conciseness from the instruction-tuned version of Gemma 9B and apply it to the base version. Figure 8c shows that increasing the steering weight shortens the outputs, demonstrating that representations learned in instruction-tuned models can still meaningfully modulate the output in the base model's space.

This is the first demonstration that cross-model steering–using vectors from instruction-tuned models–can outperform same-model steering in base models. This finding suggests that specialized representations from fine-tuned models can be leveraged to steer base models more effectively, opening new possibilities for composable transfer learning in instruction-based tasks where task vectors may originate from different models that are instruction-tuned in specialized domains.